PYTHON FOR ACTUARIES · NOTEBOOK 03

NumPy and Pandas for Actuaries

A 90-Minute Practical Workshop

In this session, you will learn to work with NumPy arrays and to explore, filter and aggregate data with Pandas.

- **Part 1**: NumPy Basics
- **Part 2**: Pandas Basics
- **Part 3**: Working with Data

Data in This Notebook

> **Explanations** use small tables created directly in the code.
>
> **Exercises** use the file `vehicle_fraud.csv` with 10,050 synthetic motor insurance claims.

```
# Import required libraries
import numpy as np
import pandas as pd

# Avoid truncation of DataFrame output in Jupyter
pd.set_option('display.expand_frame_repr', False)

# Display all columns in the DataFrame output
pd.set_option('display.max_columns', None)

# Define a separator for better readability
sep = "=" * 50
```

Part 1: NumPy Basics

NumPy provides fast arrays and vectorized operations. Pandas is built on top of it.

1.1 Creating Arrays

Dimension	Example
1D	`np.array([1, 2, 3])`
2D	`np.array([[1, 2], [3, 4]])`

A single multiplication applies a 10% loading to every premium:

$$\text{Loaded Premium} = \text{Base Premium} \times 1.10$$

```python
# Create a NumPy array of base premiums
base_premiums = np.array([500, 750, 1000, 1200, 950])

print(f"\n{sep}\nBase Premiums: base_premiums\n{sep}")
print(base_premiums)
```

```
==================================================
Base Premiums: base_premiums
==================================================
[ 500  750 1000 1200  950]
```

```python
# Apply a 10% loading
loaded_premiums = base_premiums * 1.10

print(f"\n{sep}\nLoaded Premiums: base_premiums * 1.10\n{sep}")
print(loaded_premiums)
```

```
==================================================
Loaded Premiums: base_premiums * 1.10
==================================================
[ 550.  825. 1100. 1320. 1045.]
```

A 2D array stores several values per policyholder: base premium, loading and final premium. The attributes `shape`, `ndim` and `dtype` describe the array.

```python
# Create a 2D array (matrix) of premium data
# Rows: policyholders, Columns: base, loading, final
premium_data = np.array([
    [500, 50, 550],
    [750, 75, 825],
    [1000, 100, 1100],
    [1200, 120, 1320],
    [950, 95, 1045]
])

print(f"\n{sep}\nPremium Data (2D Array): Rows=Policyholders, Cols=Types\n{sep}")
print(premium_data)
```

```
==================================================
Premium Data (2D Array): Rows=Policyholders, Cols=Types
==================================================
[[ 500   50  550]
 [ 750   75  825]
 [1000  100 1100]
 [1200  120 1320]
 [ 950   95 1045]]
```

```
# Array Attributes
print(f"\n{sep}\nArray Attributes: shape, ndim, dtype\n{sep}")
print(f"premium_data.shape: {premium_data.shape}")
print(f"premium_data.ndim:  {premium_data.ndim}")
print(f"premium_data.dtype: {premium_data.dtype}")
```

```
================================================
Array Attributes: shape, ndim, dtype
================================================
premium_data.shape: (5, 3)
premium_data.ndim:  2
premium_data.dtype: int64
```

1.2 Indexing and Slicing

Indexing starts at `0`. In a slice `start:stop`, `stop` is **not** included.

```
arr = np.array([10, 20, 30, 40, 50])
arr[0]        # 10
arr[-1]       # 50
arr[1:4]      # [20, 30, 40]
```

For 2D arrays: `arr[row, column]`. A colon `:` means "all rows" or "all columns".

> **Watch out:** `arr[row, column]` only works with NumPy arrays. A Python list of lists raises a `TypeError` for `[0, 2]`.

```
# Access Individual Element (Row 0, Col 2)
print(f"\n{sep}\nPremium Data: Rows=Policyholders, Cols=Types\n{sep}")
print(premium_data)

print(f"\n{sep}\nFirst Policyholder's Final Premium: premium_data[0, 2]\n{sep}")
print(premium_data[0, 2])
```

```
================================================
Premium Data: Rows=Policyholders, Cols=Types
================================================
[[ 500   50  550]
 [ 750   75  825]
 [1000  100 1100]
 [1200  120 1320]
 [ 950   95 1045]]

================================================
First Policyholder's Final Premium: premium_data[0, 2]
================================================
550
```

```python
# Access Column (All Final Premiums)
print(f"\n{sep}\nPremium Data: Rows=Policyholders, Cols=Types\n{sep}")
print(premium_data)

print(f"\n{sep}\nAll Final Premiums: premium_data[:, 2]\n{sep}")
print(premium_data[:, 2])
```

```
==================================================
Premium Data: Rows=Policyholders, Cols=Types
==================================================
[[ 500   50  550]
 [ 750   75  825]
 [1000  100 1100]
 [1200  120 1320]
 [ 950   95 1045]]

==================================================
All Final Premiums: premium_data[:, 2]
==================================================
[ 550  825 1100 1320 1045]
```

```python
# Access Row (First Policyholder's Data)
print(f"\n{sep}\nPremium Data: Rows=Policyholders, Cols=Types\n{sep}")
print(premium_data)

print(f"\n{sep}\nFirst Policyholder's Data: premium_data[0, :]\n{sep}")
print(premium_data[0, :])
```

```
==================================================
Premium Data: Rows=Policyholders, Cols=Types
==================================================
[[ 500   50  550]
 [ 750   75  825]
 [1000  100 1100]
 [1200  120 1320]
 [ 950   95 1045]]

==================================================
First Policyholder's Data: premium_data[0, :]
==================================================
[500  50 550]
```

```python
# Slicing (Rows 1 and 2, All Columns)
print(f"\n{sep}\nPremium Data: Rows=Policyholders, Cols=Types\n{sep}")
print(premium_data)

print(f"\n{sep}\nSecond and Third Policyholder: premium_data[1:3, :]\n{sep}")
print(premium_data[1:3, :])
```

```
==================================================
Premium Data: Rows=Policyholders, Cols=Types
==================================================
[[ 500   50  550]
 [ 750   75  825]
 [1000  100 1100]
 [1200  120 1320]
 [ 950   95 1045]]

==================================================
Second and Third Policyholder: premium_data[1:3, :]
==================================================
[[ 750   75  825]
 [1000  100 1100]]
```

1.3 Vectorized Operations and Universal Functions

Operators like `+`, `*` or `==` and universal functions like `np.sqrt()`, `np.exp()` and `np.log()` work element-wise — no loop needed.

> **Watch out:** `np.log(0)` returns `-inf` with a warning.

```python
# Element-wise arithmetic: base premium + loading
base = premium_data[:, 0]
loading = premium_data[:, 1]

print(f"\n{sep}\nBase Premiums: premium_data[:, 0]\n{sep}")
print(base)

print(f"\n{sep}\nLoadings: premium_data[:, 1]\n{sep}")
print(loading)

print(f"\n{sep}\nBase + Loading: base + loading\n{sep}")
print(base + loading)
```

```
==================================================
Base Premiums: premium_data[:, 0]
==================================================
[ 500  750 1000 1200  950]

==================================================
Loadings: premium_data[:, 1]
==================================================
[ 50  75 100 120  95]

==================================================
Base + Loading: base + loading
==================================================
[ 550  825 1100 1320 1045]
```

```python
# Check against the final premium
final = premium_data[:, 2]

print(f"\n{sep}\nBase + Loading: base + loading\n{sep}")
print(base + loading)

print(f"\n{sep}\nFinal Premiums: premium_data[:, 2]\n{sep}")
print(final)

print(f"\n{sep}\nCheck Against Final Premium: base + loading == final\n{sep}")
print(base + loading == final)
```

```
==================================================
Base + Loading: base + loading
==================================================
[ 550  825 1100 1320 1045]

==================================================
Final Premiums: premium_data[:, 2]
==================================================
[ 550  825 1100 1320 1045]

==================================================
Check Against Final Premium: base + loading == final
==================================================
[ True  True  True  True  True]
```

```python
# Universal function: natural logarithm
print(f"\n{sep}\nBase Premiums: base_premiums\n{sep}")
print(base_premiums)

print(f"\n{sep}\nNatural Logarithm: np.log(base_premiums)\n{sep}")
print(np.log(base_premiums))
```

```
==================================================
Base Premiums: base_premiums
==================================================
[ 500  750 1000 1200  950]

==================================================
Natural Logarithm: np.log(base_premiums)
==================================================
[6.2146081  6.62007321 6.90775528 7.09007684 6.85646198]
```

```python
# Universal function: square root
print(f"\n{sep}\nBase Premiums: base_premiums\n{sep}")
print(base_premiums)

print(f"\n{sep}\nSquare Root: np.sqrt(base_premiums)\n{sep}")
print(np.sqrt(base_premiums))
```

```
==================================================
Base Premiums: base_premiums
==================================================
[ 500  750 1000 1200  950]

==================================================
Square Root: np.sqrt(base_premiums)
==================================================
[22.36067977 27.38612788 31.6227766  34.64101615 30.82207001]
```

1.4 Aggregations

`arr.sum()`, `arr.mean()`, `arr.min()`, `arr.max()` reduce an array to a single number. For 2D arrays, `axis=0` gives one result per column, `axis=1` one result per row.

```
# Aggregations on a 1D array
print(f"\n{sep}\nBase Premiums: base_premiums\n{sep}")
print(base_premiums)

print(f"\n{sep}\nAggregations: base_premiums\n{sep}")
print(f"base_premiums.sum():  {base_premiums.sum()}")
print(f"base_premiums.mean(): {base_premiums.mean()}")
print(f"base_premiums.min():  {base_premiums.min()}")
print(f"base_premiums.max():  {base_premiums.max()}")
```

```
================================================
Base Premiums: base_premiums
================================================
[ 500  750 1000 1200  950]

================================================
Aggregations: base_premiums
================================================
base_premiums.sum():  4400
base_premiums.mean(): 880.0
base_premiums.min():  500
base_premiums.max():  1200
```

```
# Column totals of a 2D array
print(f"\n{sep}\nPremium Data: Rows=Policyholders, Cols=Types\n{sep}")
print(premium_data)

print(f"\n{sep}\nColumn Totals (Base, Loading, Final): premium_data.sum(axis=0)\n{sep}")
print(premium_data.sum(axis=0))
```

```
================================================
Premium Data: Rows=Policyholders, Cols=Types
================================================
[[ 500   50  550]
 [ 750   75  825]
 [1000  100 1100]
 [1200  120 1320]
 [ 950   95 1045]]

================================================
Column Totals (Base, Loading, Final): premium_data.sum(axis=0)
================================================
[4400  440 4840]
```

```python
# Row totals of a 2D array
print(f"\n{sep}\nPremium Data: Rows=Policyholders, Cols=Types\n{sep}")
print(premium_data)

print(f"\n{sep}\nRow Totals (One per Policyholder): premium_data.sum(axis=1)\n{sep}")
print(premium_data.sum(axis=1))
```

```
==================================================
Premium Data: Rows=Policyholders, Cols=Types
==================================================
[[ 500   50  550]
 [ 750   75  825]
 [1000  100 1100]
 [1200  120 1320]
 [ 950   95 1045]]

==================================================
Row Totals (One per Policyholder): premium_data.sum(axis=1)
==================================================
[1100 1650 2200 2640 2090]
```

1.5 Boolean Masks

A comparison returns a **boolean mask** of `True` / `False` values. `arr[mask]` keeps the elements where the mask is `True`; `mask.sum()` counts them. Combine conditions with `&` (and) and `|` (or) — each condition in parentheses.

```python
# Boolean mask: which premiums are above 900?
mask = base_premiums > 900

print(f"\n{sep}\nBase Premiums: base_premiums\n{sep}")
print(base_premiums)

print(f"\n{sep}\nBoolean Mask: base_premiums > 900\n{sep}")
print(mask)
```

```
==================================================
Base Premiums: base_premiums
==================================================
[ 500  750 1000 1200  950]

==================================================
Boolean Mask: base_premiums > 900
==================================================
[False False  True  True  True]
```

```python
# Keep the premiums where the mask is True
print(f"\n{sep}\nBase Premiums: base_premiums\n{sep}")
print(base_premiums)

print(f"\n{sep}\nBoolean Mask: base_premiums > 900\n{sep}")
print(mask)

print(f"\n{sep}\nSelected Premiums: base_premiums[mask]\n{sep}")
print(base_premiums[mask])
```

```
==================================================
Base Premiums: base_premiums
==================================================
[ 500  750 1000 1200  950]

==================================================
Boolean Mask: base_premiums > 900
==================================================
[False False  True  True  True]

==================================================
Selected Premiums: base_premiums[mask]
==================================================
[1000 1200  950]
```

```python
# Count the True values
print(f"\n{sep}\nBoolean Mask: base_premiums > 900\n{sep}")
print(mask)

print(f"\n{sep}\nNumber of Premiums Above 900: mask.sum()\n{sep}")
print(mask.sum())
```

```
==================================================
Boolean Mask: base_premiums > 900
==================================================
[False False  True  True  True]

==================================================
Number of Premiums Above 900: mask.sum()
==================================================
3
```

```python
# Combining conditions
print(f"\n{sep}\nBase Premiums: base_premiums\n{sep}")
print(base_premiums)

print(f"\n{sep}\nPremiums Between 600 and 1100: (base_premiums > 600) & (base_premiums <
    1100)\n{sep}")
print(base_premiums[(base_premiums > 600) & (base_premiums < 1100)])
```

```
==================================================
Base Premiums: base_premiums
==================================================
[ 500  750 1000 1200  950]

==================================================
Premiums Between 600 and 1100: (base_premiums > 600) & (base_premiums < 1100)
==================================================
[ 750 1000  950]
```

Exercise 1: NumPy on `vehicle_fraud.csv`

Analyze claim amounts and vehicle values as NumPy arrays: statistics, boolean masks and logarithms.

The first code cell loads two columns as NumPy arrays (Pandas follows in Part 2).

```
# Load two columns of vehicle_fraud.csv as NumPy arrays
df = pd.read_csv('vehicle_fraud.csv')
claims = df['claim_amount'].to_numpy()
values = df['vehicle_value'].to_numpy()
```

Task 1: How many claims are there (`.shape`)? Print the first five claim amounts.

```
# Task 1: Number of claims and first five claim amounts
# Hint: claims.shape, claims[:5]
```

Task 2: Apply a 10% inflation to all claim amounts.

```
# Task 2: 10% inflation
# Hint: claims * 1.10
```

Task 3: Compute the mean, minimum and maximum claim amount.

```
# Task 3: Mean, minimum, maximum
# Hint: claims.mean(), claims.min(), claims.max()
```

Task 4: How many claims are above 20,000? What is their share?

```
# Task 4: Claims above 20,000
# Hint: mask = claims > 20000
#       mask.sum() counts the True values, mask.mean() gives their share
```

Task 5: Compute the claim-to-value ratio `claims / values` for every claim. How many claims exceed the vehicle value?

```
# Task 5: Claim-to-value ratio
# Hint: ratio = claims / values
#       (ratio > 1).sum()
```

Task 6: Compute `np.log(claims)`. Compare minimum and maximum before and after the logarithm.

```
# Task 6: Logarithm of the claim amounts
# Hint: log_claims = np.log(claims)
#       log_claims.min(), log_claims.max()
```

Part 2: Pandas Basics

2.1 Series

A Series is a one-dimensional labeled array. Without a custom `index`, the labels are `0, 1, 2, ...`.

```
# Series with automatic integer index
age_series = pd.Series([28, 35, 42], name='age')

print(f"\n{sep}\nSeries with Default Index: pd.Series([28, 35, 42], name='age')\n{sep}")
print(age_series)
```

```
==================================================
Series with Default Index: pd.Series([28, 35, 42], name='age')
==================================================
0    28
1    35
2    42
Name: age, dtype: int64
```

```
# Series with a custom index
claim_series = pd.Series([1200, 5400, 800], index=['C101', 'C102', 'C103'],
    name='claim_amount')

print(f"\n{sep}\nSeries with Custom Index: index=['C101', 'C102', 'C103']\n{sep}")
print(claim_series)
```

```
==================================================
Series with Custom Index: index=['C101', 'C102', 'C103']
==================================================
C101    1200
C102    5400
C103     800
Name: claim_amount, dtype: int64
```

2.2 DataFrames

A DataFrame is a table: each dictionary key becomes a column name, each list a column. Every column is a Series.

The table `claims` with eight claims is used for all explanations in Parts 2 and 3.

```
# Create a small DataFrame from a dictionary
claims = pd.DataFrame({
    'claim_id': [101, 102, 103, 104, 105, 106, 107, 108],
    'age': [23, 35, 47, 52, 61, 29, 44, 68],
    'region': ['North', 'South', 'North', 'East', 'South', 'North', 'East', 'South'],
    'vehicle_type': ['Compact', 'SUV', 'Compact', 'Luxury', 'SUV', 'Compact', 'Luxury',
        'Compact'],
    'claim_amount': [1200.0, 5400.0, 800.0, 15000.0, 2300.0, 9800.0, 4100.0, 650.0],
    'police_report': [1, 0, 1, 1, 0, 0, 1, 1],
    'fraud': [0, 1, 0, 0, 0, 1, 0, 0]
})

print(f"\n{sep}\nSmall DataFrame: pd.DataFrame({{...}})\n{sep}")
print(claims)
```

```
================================================
Small DataFrame: pd.DataFrame({...})
================================================
   claim_id  age region vehicle_type  claim_amount  police_report  fraud
0       101   23  North      Compact        1200.0              1      0
1       102   35  South          SUV        5400.0              0      1
2       103   47  North      Compact         800.0              1      0
3       104   52   East       Luxury       15000.0              1      0
4       105   61  South          SUV        2300.0              0      0
5       106   29  North      Compact        9800.0              0      1
6       107   44   East       Luxury        4100.0              1      0
7       108   68  South      Compact         650.0              1      0
```

```
# Each column is a Series
print(f"\n{sep}\nType of a Column: type(claims['age'])\n{sep}")
print(type(claims['age']))
```

```
================================================
Type of a Column: type(claims['age'])
================================================
<class 'pandas.Series'>
```

Displaying DataFrames

Command	Output
`print(claims)`	Plain text, anywhere in the cell
`display(claims)`	Formatted table, anywhere in the cell
`claims` as the last line	Formatted table, only for the last line of a cell

This notebook uses `print()` with a header line, so that every output shows the code that produced it.

```
# Plain text output
print(claims)
```

```
   claim_id  age region vehicle_type  claim_amount  police_report  fraud
0       101   23  North      Compact        1200.0              1      0
1       102   35  South          SUV        5400.0              0      1
2       103   47  North      Compact         800.0              1      0
3       104   52   East       Luxury       15000.0              1      0
4       105   61  South          SUV        2300.0              0      0
5       106   29  North      Compact        9800.0              0      1
6       107   44   East       Luxury        4100.0              1      0
7       108   68  South      Compact         650.0              1      0
```

```
# Formatted table
display(claims)
```

	claim_id	age	region	vehicle_type	claim_amount	police_report	fraud
0	101	23	North	Compact	1200.0	1	0
1	102	35	South	SUV	5400.0	0	1
2	103	47	North	Compact	800.0	1	0
3	104	52	East	Luxury	15000.0	1	0
4	105	61	South	SUV	2300.0	0	0
5	106	29	North	Compact	9800.0	0	1
6	107	44	East	Luxury	4100.0	1	0
7	108	68	South	Compact	650.0	1	0

```
# Last line of a cell
claims
```

	claim_id	age	region	vehicle_type	claim_amount	police_report	fraud
0	101	23	North	Compact	1200.0	1	0
1	102	35	South	SUV	5400.0	0	1
2	103	47	North	Compact	800.0	1	0
3	104	52	East	Luxury	15000.0	1	0
4	105	61	South	SUV	2300.0	0	0
5	106	29	North	Compact	9800.0	0	1
6	107	44	East	Luxury	4100.0	1	0
7	108	68	South	Compact	650.0	1	0

2.3 Writing and Reading CSV Files

`claims.to_csv('file.csv', index=False)` writes a DataFrame to a file. `index=False` omits the row numbers; keep the index if it carries information, e.g. the regions after a `groupby()`.

`pd.read_csv()` reads a CSV file into a DataFrame.

```
# Write the table to a CSV file
claims.to_csv('claims.csv', index=False)
print('Data written to claims.csv')
```

```
Data written to claims.csv
```

```
# Read the CSV file back into a new DataFrame
claims_loaded = pd.read_csv('claims.csv')
```

```
# Number of rows and columns
print(f"\n{sep}\nShape (Rows, Columns): claims_loaded.shape\n{sep}")
print(claims_loaded.shape)
```

```
==================================================
Shape (Rows, Columns): claims_loaded.shape
==================================================
(8, 7)
```

```
# Preview
print(f"\n{sep}\nData Preview: claims_loaded.head()\n{sep}")
print(claims_loaded.head())
```

```
==================================================
Data Preview: claims_loaded.head()
==================================================
   claim_id  age region vehicle_type  claim_amount  police_report  fraud
0       101   23  North      Compact        1200.0              1      0
1       102   35  South          SUV        5400.0              0      1
2       103   47  North      Compact         800.0              1      0
3       104   52   East       Luxury       15000.0              1      0
4       105   61  South          SUV        2300.0              0      0
```

2.4 Accessing Data

Syntax	Purpose	Example
`df['column']`	One column	`claims['age']`
`df[['col1', 'col2']]`	Several columns	`claims[['age', 'region']]`
`.loc[row, column]`	By **label**	`claims.loc[0, 'age']`
`.iloc[row, column]`	By **position**	`claims.iloc[-1, 1]`
`.to_numpy()`	Column as NumPy array	`claims['age'].to_numpy()`

> **Watch out:** In a `.loc` slice, the end label **is** included: `claims.loc[0:2, :]` returns 3 rows.

```
# Single value by label: row 0, column 'age'
print(f"\n{sep}\nSingle Value by Label: claims.loc[0, 'age']\n{sep}")
print(claims.loc[0, 'age'])
```

```
==================================================
Single Value by Label: claims.loc[0, 'age']
==================================================
23
```

```python
# Single value by position: last row, second column
print(f"\n{sep}\nSingle Value by Position: claims.iloc[-1, 1]\n{sep}")
print(claims.iloc[-1, 1])
```

```
==================================================
Single Value by Position: claims.iloc[-1, 1]
==================================================
68
```

```python
# Column Access
print(f"\n{sep}\nColumn Access: claims['age']\n{sep}")
print(claims['age'])
```

```
==================================================
Column Access: claims['age']
==================================================
0    23
1    35
2    47
3    52
4    61
5    29
6    44
7    68
Name: age, dtype: int64
```

```python
# One Column as DataFrame
print(f"\n{sep}\nOne Column as DataFrame: claims[['age']]\n{sep}")
print(claims[['age']])
```

```
==================================================
One Column as DataFrame: claims[['age']]
==================================================
   age
0   23
1   35
2   47
3   52
4   61
5   29
6   44
7   68
```

> **Single vs. double brackets:** `claims['age']` returns a **Series**. `claims[['age']]` returns a **DataFrame** with one column, because the inner brackets are a list of column names.

```
# Several Columns
print(f"\n{sep}\nSeveral Columns: claims[['age', 'region']]\n{sep}")
print(claims[['age', 'region']])
```

```
==================================================
Several Columns: claims[['age', 'region']]
==================================================
   age region
0   23  North
1   35  South
2   47  North
3   52   East
4   61  South
5   29  North
6   44   East
7   68  South
```

```
# Row Access
print(f"\n{sep}\nRow Access: claims.loc[0, :]\n{sep}")
print(claims.loc[0, :])
```

```
==================================================
Row Access: claims.loc[0, :]
==================================================
claim_id             101
age                   23
region             North
vehicle_type     Compact
claim_amount      1200.0
police_report          1
fraud                  0
Name: 0, dtype: object
```

```
# Slicing with .loc: end label is included -> 3 rows
print(f"\n{sep}\nSlicing: claims.loc[0:2, ['age', 'region']]\n{sep}")
print(claims.loc[0:2, ['age', 'region']])
```

```
==================================================
Slicing: claims.loc[0:2, ['age', 'region']]
==================================================
   age region
0   23  North
1   35  South
2   47  North
```

2.5 Exploring Data

```
df.head()                       # First 5 rows
df.tail()                       # Last 5 rows
df.info()                       # Data types and non-null counts
df.describe()                   # Statistics of the numeric columns
df.columns                      # Column names
df.dtypes                       # Data types
df.isna().sum()                 # Missing values per column
df['column'].unique().tolist()  # Unique values
df['column'].value_counts()     # Value frequencies
```

```python
# Dataset Information: claims.info()
print(f"\n{sep}\nDataset Information: claims.info()\n{sep}")
claims.info()
```

```
==================================================
Dataset Information: claims.info()
==================================================
<class 'pandas.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 7 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   claim_id       8 non-null      int64
 1   age            8 non-null      int64
 2   region         8 non-null      str
 3   vehicle_type   8 non-null      str
 4   claim_amount   8 non-null      float64
 5   police_report  8 non-null      int64
 6   fraud          8 non-null      int64
dtypes: float64(1), int64(4), str(2)
memory usage: 580.0 bytes
```

```python
# Statistical Summary: claims.describe()
print(f"\n{sep}\nStatistical Summary: claims.describe()\n{sep}")
print(claims.describe())
```

```
==================================================
Statistical Summary: claims.describe()
==================================================
        claim_id        age  claim_amount  police_report    fraud
count    8.00000   8.000000      8.000000       8.000000  8.00000
mean   104.50000  44.875000   4906.250000       0.625000  0.25000
std      2.44949  15.486745   5102.271028       0.517549  0.46291
min    101.00000  23.000000    650.000000       0.000000  0.00000
25%    102.75000  33.500000   1100.000000       0.000000  0.00000
50%    104.50000  45.500000   3200.000000       1.000000  0.00000
75%    106.25000  54.250000   6500.000000       1.000000  0.25000
max    108.00000  68.000000  15000.000000       1.000000  1.00000
```

`describe()` also summarizes `claim_id` — but the mean of an ID is meaningless.

```python
# Column Names
print(f"\n{sep}\nColumn Names: claims.columns.tolist()\n{sep}")
print(claims.columns.tolist())
```

```
==================================================
Column Names: claims.columns.tolist()
==================================================
['claim_id', 'age', 'region', 'vehicle_type', 'claim_amount', 'police_report', 'fraud']
```

```python
# Data Types
print(f"\n{sep}\nData Types: claims.dtypes\n{sep}")
print(claims.dtypes)
```

```
================================================
Data Types: claims.dtypes
================================================
claim_id           int64
age                int64
region               str
vehicle_type         str
claim_amount     float64
police_report      int64
fraud              int64
dtype: object
```

```python
# Missing Values
print(f"\n{sep}\nMissing Value Mask: claims.isna()\n{sep}")
print(claims.isna())

print(f"\n{sep}\nMissing Values: claims.isna().sum()\n{sep}")
print(claims.isna().sum())
```

```
================================================
Missing Value Mask: claims.isna()
================================================
   claim_id    age  region  vehicle_type  claim_amount  police_report  fraud
0     False  False   False         False         False          False  False
1     False  False   False         False         False          False  False
2     False  False   False         False         False          False  False
3     False  False   False         False         False          False  False
4     False  False   False         False         False          False  False
5     False  False   False         False         False          False  False
6     False  False   False         False         False          False  False
7     False  False   False         False         False          False  False

================================================
Missing Values: claims.isna().sum()
================================================
claim_id         0
age              0
region           0
vehicle_type     0
claim_amount     0
police_report    0
fraud            0
dtype: int64
```

The table has no missing values — real-world data usually does.

```python
# Unique Values: Region
print(f"\n{sep}\nUnique Regions: claims['region'].unique().tolist()\n{sep}")
print(claims['region'].unique().tolist())
```

```
================================================
Unique Regions: claims['region'].unique().tolist()
================================================
['North', 'South', 'East']
```

```
# Distribution: Vehicle Type
print(f"\n{sep}\nVehicle Type Distribution: claims['vehicle_type'].value_counts()\n{sep}")
print(claims['vehicle_type'].value_counts())
```

```
==================================================
Vehicle Type Distribution: claims['vehicle_type'].value_counts()
==================================================
vehicle_type
Compact    4
SUV        2
Luxury     2
Name: count, dtype: int64
```

Exercise 2: Exploring `vehicle_fraud.csv`

Load the dataset and get an overview: size, vehicle types, selected rows and statistics.

Task 1: Load `vehicle_fraud.csv`. Show the last three rows. How many rows and columns are there?

```
# Task 1: Load the dataset, last three rows and shape
# Hint: df = pd.read_csv('vehicle_fraud.csv')
#       df.tail(3), df.shape
```

Task 2: Which vehicle types occur, how often, and with which share?

```
# Task 2: Vehicle types and their shares
# Hint: df['vehicle_type'].value_counts()
#       df['vehicle_type'].value_counts(normalize=True)
```

Task 3: Show the first three rows and the first three columns with `.iloc`.

```
# Task 3: First three rows and columns
# Hint: df.iloc[:3, :3]
```

Task 4: Show `policyholder_age`, `vehicle_type` and `claim_amount` for the row labels `10` to `14`.

```
# Task 4: Rows 10 to 14, three columns
# Hint: df.loc[10:14, [...]]
```

Task 5: Use `describe()` on `claim_amount`. What does the difference between mean and median (`50%`) tell you?

```
# Task 5: Statistics of the claim amounts
# Hint: df['claim_amount'].describe()
```

Part 3: Working with Data

3.1 Filtering Rows

Filtering works like boolean masks in NumPy: `df[mask]` keeps the rows where the mask is `True`, `len()` counts them.

```
claims[claims['age'] > 60]
claims[(claims['region'] == 'North') & (claims['claim_amount'] > 1000)]
claims[claims['vehicle_type'].isin(['SUV', 'Luxury'])]
```

```
# The mask itself: a Series of True/False values
print(f"\n{sep}\nBoolean Mask: claims['age'] > 60\n{sep}")
print(claims['age'] > 60)
```

```
==================================================
Boolean Mask: claims['age'] > 60
==================================================
0    False
1    False
2    False
3    False
4     True
5    False
6    False
7     True
Name: age, dtype: bool
```

```
# Filter: policyholders older than 60
claims_senior = claims[claims['age'] > 60]

print(f"\n{sep}\nPolicyholders Older Than 60: claims[claims['age'] > 60]\n{sep}")
print(claims_senior)
print(f"Number of rows: {len(claims_senior)}")
```

```
==================================================
Policyholders Older Than 60: claims[claims['age'] > 60]
==================================================
   claim_id  age region vehicle_type  claim_amount  police_report  fraud
4       105   61  South          SUV        2300.0              0      0
7       108   68  South      Compact         650.0              1      0
Number of rows: 2
```

```
# Combined filter: region North AND claim amount above 1000
claims_north_high = claims[(claims['region'] == 'North') & (claims['claim_amount'] > 1000)]

print(f"\n{sep}\nNorth With Claim Amount Above 1000: claims[(claims['region'] == 'North') &
    (claims['claim_amount'] > 1000)]\n{sep}")
print(claims_north_high)
```

```
==================================================
North With Claim Amount Above 1000: claims[(claims['region'] == 'North') &
(claims['claim_amount'] > 1000)]
==================================================
   claim_id  age region vehicle_type  claim_amount  police_report  fraud
0       101   23  North      Compact        1200.0              1      0
5       106   29  North      Compact        9800.0              0      1
```

```python
# Filter with a list of values
claims_large_cars = claims[claims['vehicle_type'].isin(['SUV', 'Luxury'])]

print(f"\n{sep}\nSUV or Luxury: claims[claims['vehicle_type'].isin(['SUV',
    'Luxury'])]\n{sep}")
print(claims_large_cars)
print(f"Number of rows: {len(claims_large_cars)}")
```

```
================================================
SUV or Luxury: claims[claims['vehicle_type'].isin(['SUV', 'Luxury'])]
================================================
   claim_id  age region vehicle_type  claim_amount  police_report  fraud
1       102   35  South          SUV        5400.0              0      1
3       104   52   East       Luxury       15000.0              1      0
4       105   61  South          SUV        2300.0              0      0
6       107   44   East       Luxury        4100.0              1      0
Number of rows: 4
```

3.2 Creating New Columns

Assign to a new column name: `claims['inflated_amount'] = claims['claim_amount'] * 1.10`.

`pd.cut()` divides a numeric column into groups. With `right=False`, each interval includes its left edge: a 30-year-old falls into `30-39`.

```python
# New column from a calculation
claims['inflated_amount'] = claims['claim_amount'] * 1.10

print(f"\n{sep}\nNew Column: inflated_amount\n{sep}")
print(claims[['claim_amount', 'inflated_amount']])
```

```
================================================
New Column: inflated_amount
================================================
   claim_amount  inflated_amount
0        1200.0           1320.0
1        5400.0           5940.0
2         800.0            880.0
3       15000.0          16500.0
4        2300.0           2530.0
5        9800.0          10780.0
6        4100.0           4510.0
7         650.0            715.0
```

```
# New column from a condition (True/False)
claims['large_claim'] = claims['claim_amount'] > 5000

print(f"\n{sep}\nNew Column: large_claim\n{sep}")
print(claims[['claim_amount', 'large_claim']])
```

```
================================================
New Column: large_claim
================================================
   claim_amount  large_claim
0        1200.0        False
1        5400.0         True
2         800.0        False
3       15000.0         True
4        2300.0        False
5        9800.0         True
6        4100.0        False
7         650.0        False
```

```
# Age Group Binning: pd.cut()
# right=False: the left edge belongs to the group, so age 30 goes to '30-39' and age 40 to
    '40-49'
claims['age_group'] = pd.cut(claims['age'], bins=[0, 30, 40, 50, 100], right=False,
                             labels=['<30', '30-39', '40-49', '50+'])

print(f"\n{sep}\nAge Groups: claims[['age', 'age_group']]\n{sep}")
print(claims[['age', 'age_group']])
```

```
================================================
Age Groups: claims[['age', 'age_group']]
================================================
   age age_group
0   23       <30
1   35     30-39
2   47     40-49
3   52       50+
4   61       50+
5   29       <30
6   44     40-49
7   68       50+
```

3.3 Sorting

`claims.sort_values('claim_amount')` sorts the rows by a column, from smallest to largest. `ascending=False` sorts from largest to smallest. The row labels move with their rows.

A Series is sorted with `.sort_values()` without a column name.

```
# Sort rows by claim amount, smallest first
print(f"\n{sep}\nClaims: claims[['claim_id', 'region', 'claim_amount']]\n{sep}")
print(claims[['claim_id', 'region', 'claim_amount']])

print(f"\n{sep}\nSorted by Claim Amount: claims[['claim_id', 'region',
    'claim_amount']].sort_values('claim_amount')\n{sep}")
print(claims[['claim_id', 'region', 'claim_amount']].sort_values('claim_amount'))
```

```
================================================
Claims: claims[['claim_id', 'region', 'claim_amount']]
================================================
   claim_id region  claim_amount
```

```
0       101  North        1200.0
1       102  South        5400.0
2       103  North         800.0
3       104   East       15000.0
4       105  South        2300.0
5       106  North        9800.0
6       107   East        4100.0
7       108  South         650.0

================================================
Sorted by Claim Amount: claims[['claim_id', 'region',
'claim_amount']].sort_values('claim_amount')
================================================
   claim_id region  claim_amount
7       108  South         650.0
2       103  North         800.0
0       101  North        1200.0
4       105  South        2300.0
6       107   East        4100.0
1       102  South        5400.0
5       106  North        9800.0
3       104   East       15000.0
```

```python
# Sort rows by claim amount, largest first
print(f"\n{sep}\nClaims: claims[['claim_id', 'region', 'claim_amount']]\n{sep}")
print(claims[['claim_id', 'region', 'claim_amount']])

print(f"\n{sep}\nSorted Descending: claims[['claim_id', 'region',
    'claim_amount']].sort_values('claim_amount', ascending=False)\n{sep}")
print(claims[['claim_id', 'region', 'claim_amount']].sort_values('claim_amount',
    ascending=False))
```

```
================================================
Claims: claims[['claim_id', 'region', 'claim_amount']]
================================================
   claim_id region  claim_amount
0       101  North        1200.0
1       102  South        5400.0
2       103  North         800.0
3       104   East       15000.0
4       105  South        2300.0
5       106  North        9800.0
6       107   East        4100.0
7       108  South         650.0

================================================
Sorted Descending: claims[['claim_id', 'region', 'claim_amount']].sort_values('claim_amount',
ascending=False)
================================================
   claim_id region  claim_amount
3       104   East       15000.0
5       106  North        9800.0
1       102  South        5400.0
6       107   East        4100.0
4       105  South        2300.0
0       101  North        1200.0
2       103  North         800.0
7       108  South         650.0
```

```python
# Sort a Series
print(f"\n{sep}\nAges: claims['age']\n{sep}")
print(claims['age'])

print(f"\n{sep}\nSorted Ages: claims['age'].sort_values()\n{sep}")
print(claims['age'].sort_values())
```

```
==================================================
Ages: claims['age']
==================================================
0    23
1    35
2    47
3    52
4    61
5    29
6    44
7    68
Name: age, dtype: int64

==================================================
Sorted Ages: claims['age'].sort_values()
==================================================
0    23
5    29
1    35
6    44
2    47
3    52
4    61
7    68
Name: age, dtype: int64
```

3.4 Grouping and Aggregation

Grouping splits the rows into groups that share the same value in one column. **Aggregation** then reduces each group to a single number, e.g. its mean. The result has one row per group.

A small example with four claims:

region	claim_amount
North	1200
South	5400
North	800
South	2300

Step 1 – Grouping by `region`:

Group	claim_amount
North	1200, 800
South	5400, 2300

Step 2 – Aggregation with the mean:

region	Mean claim_amount
North	(1200 + 800) / 2 = 1000

region	Mean claim_amount
South	(5400 + 2300) / 2 = 3850

In Pandas, both steps are one line: `claims.groupby('region')['claim_amount'].mean()`.

`.agg(['mean', 'median', 'count'])` applies several functions at once. When grouping by a `pd.cut()` column, pass `observed=True`.

The mean of a 0/1 column is a rate: `claims.groupby('region')['fraud'].mean()` gives the fraud rate per region.

```
# Mean claim amount by region, sorted
print(f"\n{sep}\nMean Claim Amount by Region: claims.groupby('region')
    ['claim_amount'].mean().sort_values(ascending=False)\n{sep}")
print(claims.groupby('region')['claim_amount'].mean().sort_values(ascending=False))
```

```
==================================================
Mean Claim Amount by Region: claims.groupby('region')
['claim_amount'].mean().sort_values(ascending=False)
==================================================
region
East     9550.000000
North    3933.333333
South    2783.333333
Name: claim_amount, dtype: float64
```

```
# Several aggregation functions at once
print(f"\n{sep}\nClaim Analysis by Region: claims.groupby('region')
    ['claim_amount'].agg(['mean', 'median', 'count'])\n{sep}")
print(claims.groupby('region')['claim_amount'].agg(['mean', 'median', 'count']))
```

```
==================================================
Claim Analysis by Region: claims.groupby('region')['claim_amount'].agg(['mean', 'median',
'count'])
==================================================
               mean  median  count
region
East    9550.000000  9550.0      2
North   3933.333333  1200.0      3
South   2783.333333  2300.0      3
```

```
# Mean of a 0/1 column: fraud rate
print(f"\n{sep}\nFraud Rate by Region: claims.groupby('region')['fraud'].mean()\n{sep}")
print(claims.groupby('region')['fraud'].mean())
```

```
==================================================
Fraud Rate by Region: claims.groupby('region')['fraud'].mean()
==================================================
region
East     0.000000
North    0.333333
South    0.333333
Name: fraud, dtype: float64
```

```
# Grouping by a pd.cut() column
print(f"\n{sep}\nMean Claim Amount by Age Group: claims.groupby('age_group', observed=True)
    ['claim_amount'].mean()\n{sep}")
print(claims.groupby('age_group', observed=True)['claim_amount'].mean())
```

```
================================================
Mean Claim Amount by Age Group: claims.groupby('age_group', observed=True)
['claim_amount'].mean()
================================================
age_group
<30      5500.000000
30-39    5400.000000
40-49    2450.000000
50+      5983.333333
Name: claim_amount, dtype: float64
```

With two grouping columns, the result has a two-level row index. `.unstack()` moves the inner level (`vehicle_type`) into the columns and turns the result into a compact table:

```
# Group by two columns
print(f"\n{sep}\nMean Claim Amount by Region and Vehicle Type: claims.groupby(['region',
    'vehicle_type'])['claim_amount'].mean()\n{sep}")
print(claims.groupby(['region', 'vehicle_type'])['claim_amount'].mean())

print(f"\n{sep}\nAs a Table: claims.groupby(['region', 'vehicle_type'])
    ['claim_amount'].mean().unstack()\n{sep}")
print(claims.groupby(['region', 'vehicle_type'])['claim_amount'].mean().unstack())
```

```
================================================
Mean Claim Amount by Region and Vehicle Type: claims.groupby(['region', 'vehicle_type'])
['claim_amount'].mean()
================================================
region  vehicle_type
East    Luxury          9550.000000
North   Compact         3933.333333
South   Compact          650.000000
        SUV             3850.000000
Name: claim_amount, dtype: float64

================================================
As a Table: claims.groupby(['region', 'vehicle_type'])['claim_amount'].mean().unstack()
================================================
vehicle_type      Compact  Luxury     SUV
region
East                  NaN  9550.0     NaN
North         3933.333333     NaN     NaN
South          650.000000     NaN  3850.0
```

```
# Always check the group sizes!
print(f"\n{sep}\nGroup Sizes: claims.groupby(['region', 'vehicle_type'])
    ['claim_amount'].count().unstack(fill_value=0)\n{sep}")
print(claims.groupby(['region', 'vehicle_type'])
    ['claim_amount'].count().unstack(fill_value=0))
```

```
================================================
Group Sizes: claims.groupby(['region', 'vehicle_type'])
['claim_amount'].count().unstack(fill_value=0)
================================================
vehicle_type  Compact  Luxury  SUV
region
East                0       2    0
North               3       0    0
South               1       0    2
```

`NaN` means: no data. There are no SUVs in North and East. The mean for Compact in South rests on a single claim — always check the group sizes.

Exercise 3: Working with `vehicle_fraud.csv`

Filter claims, add a ratio column, aggregate by group and save a result as a CSV file.

```
# Load the dataset
df = pd.read_csv('vehicle_fraud.csv')
```

Task 1: How many claims have no police report? Show all `Luxury` claims with a claim amount above 50,000.

```
# Task 1: Filtering
# Hint: len(df[df['police_report'] == 0])
#       df[(df['vehicle_type'] == 'Luxury') & (df['claim_amount'] > 50000)]
```

Task 2: Add a column `claim_value_ratio` = `claim_amount` / `vehicle_value`. Show the five highest values.

```
# Task 2: New column claim_value_ratio and the five highest values
# Hint: df.sort_values('claim_value_ratio', ascending=False).head()
```

Task 3: Compute the mean `number_of_prior_claims` and `claim_amount` per `vehicle_type`.

```
# Task 3: Mean number_of_prior_claims and claim_amount per vehicle_type
# Hint: df.groupby('vehicle_type')[['number_of_prior_claims', 'claim_amount']].mean()
```

Task 4: Compute the claim-to-value ratio per region from the totals of `claim_amount` and `vehicle_value`. Why does it differ from the mean of `claim_value_ratio` per region?

```
# Task 4: Claim-to-value ratio per region
# Hint: totals = df.groupby('region')[['claim_amount', 'vehicle_value']].sum()
#       totals['ratio'] = totals['claim_amount'] / totals['vehicle_value']
#       Compare with: df.groupby('region')['claim_value_ratio'].mean()
```

Task 5: Create `age_group` from `policyholder_age` as in Section 3.2 and compute the fraud rate per age group.

```
# Task 5: Fraud rate per age group
# Hint: df.groupby('age_group', observed=True)['fraud'].mean()
```

Task 6: Write the result of Task 4 to `ratio_by_region.csv`.

```
# Task 6: Export
# Hint: totals.to_csv(...)
```

Summary

- **NumPy:** arrays, indexing, vectorized operations, aggregations, boolean masks
- **Pandas:** Series and DataFrames, `to_csv()`, `read_csv()`, `.loc[]` / `.iloc[]`, exploring data
- **Working with data:** filtering, new columns, `pd.cut()`, `sort_values()`, `groupby()`

Next: visualizing data in Notebook 04.

- NumPy Documentation
- Pandas Documentation